PROYECTO DE CURSO — PROGRAMACIÓN ORIENTADA A OBJETOS

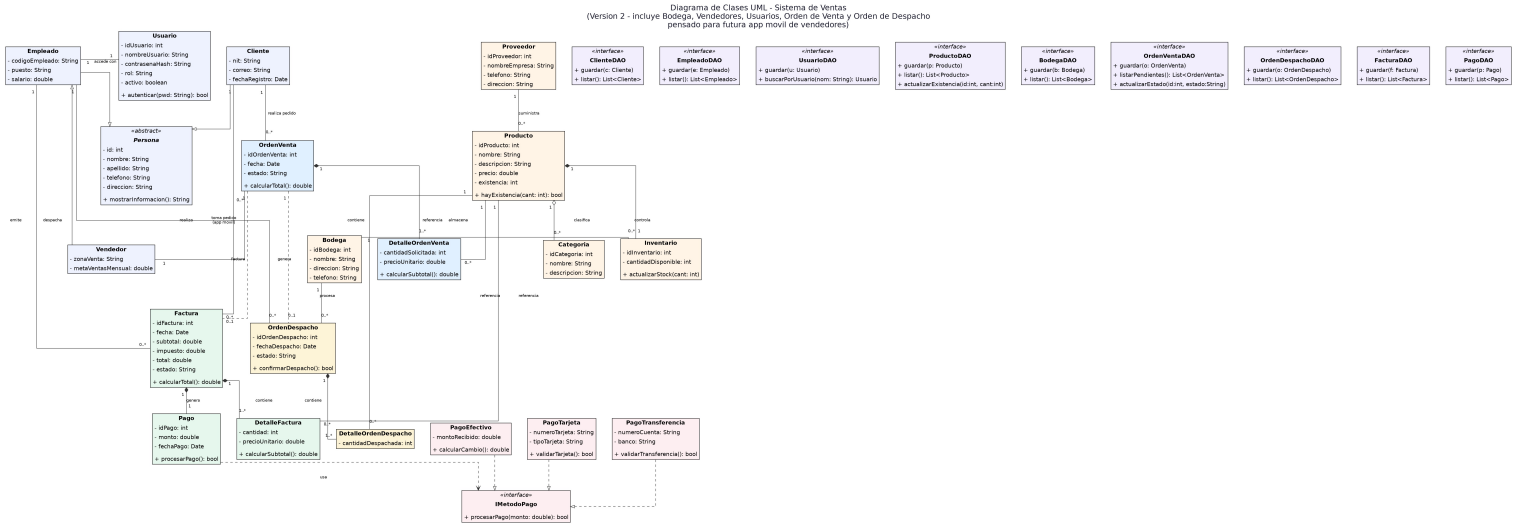
Sistema de Ventas

Diagrama de Clases UML & Organización del Equipo de Trabajo

Integrante	Módulos asignados
Anthony	Núcleo, arquitectura, integración y Usuarios (login)
Cristofer	Cliente, Empleado y Vendedor
André	Producto, Categoría, Inventario, Proveedor y Bodega
Carlos	Facturación y Orden de Venta (pedido)
Oscar	Pagos y Orden de Despacho

Arquitectura: MVC + DAO · Java + Java Swing · PostgreSQL · v2: preparado para futura app móvil de vendedores

1. Diagrama de Clases UML Completo (v2)



Versión 2: incorpora Bodega, Vendedor, Usuario, Orden de Venta y Orden de Despacho — el flujo pensado para la futura app móvil de vendedores. Las interfaces `XxxDAOImpl` no se dibujan por espacio; su detalle está en la sección 6.

> **Flujo completo del negocio (de principio a fin):** el Cliente hace un pedido con ayuda de un Vendedor (a futuro, desde su celular) → se crea una `OrdenVenta` → la Bodega la revisa y genera una `OrdenDespacho` con lo que realmente hay en existencia → una vez despachada, se genera la `Factura` → el Cliente paga con alguna `IMetodoPago` y el sistema registra el `Pago`.

2. Guía de lectura del diagrama

Personas (Cliente / Empleado) Catálogo e inventario Facturación y pago Estrategias de pago Capa DAO (persistencia)

Símbolo	Tipo de relación	Dónde aparece en el diagrama
Triángulo hueco, línea sólida	Herencia	`Persona` → `Cliente`, `Empleado`
Línea simple con multiplicidad	Asociación	Cliente–Factura, Empleado–Factura, Proveedor–Producto, DetalleFactura–Producto
Diamante hueco	Agregación	Categoria ◇— Producto (el producto puede existir sin la categoría)
Diamante relleno	Composición	Factura ◆— DetalleFactura, Producto ◆— Inventario, Factura ◆— Pago (no existen sin su "todo")
Línea punteada + triángulo hueco	Realización (implementa interfaz)	PagoEfectivo/PagoTarjeta/PagoTransferencia → IMetodoPago; XxxDAOImpl → XxxDAO
Línea punteada + flecha	Dependencia	Pago usa IMetodoPago; cada DAOImpl gestiona su entidad
Línea punteada + flecha abierta	Asociación dirigida (navegación de un solo sentido)	OrdenVenta → OrdenDespacho ("genera"), OrdenVenta → Factura ("factura") — el pedido no conoce su despacho o factura hasta que existen

> **Nota sobre las interfaces:** cada entidad principal (Cliente, Empleado, Producto, Factura, Pago) tiene su propia interfaz DAO (`ClienteDAO`, `ProductoDAO`, etc.) con una clase `XxxDAOImpl` que la implementa y contiene el código SQL real. Esto sigue el mismo patrón usado en el proyecto de referencia del curso y cumple con el requisito de incluir interfaces de repositorio en el diagrama.

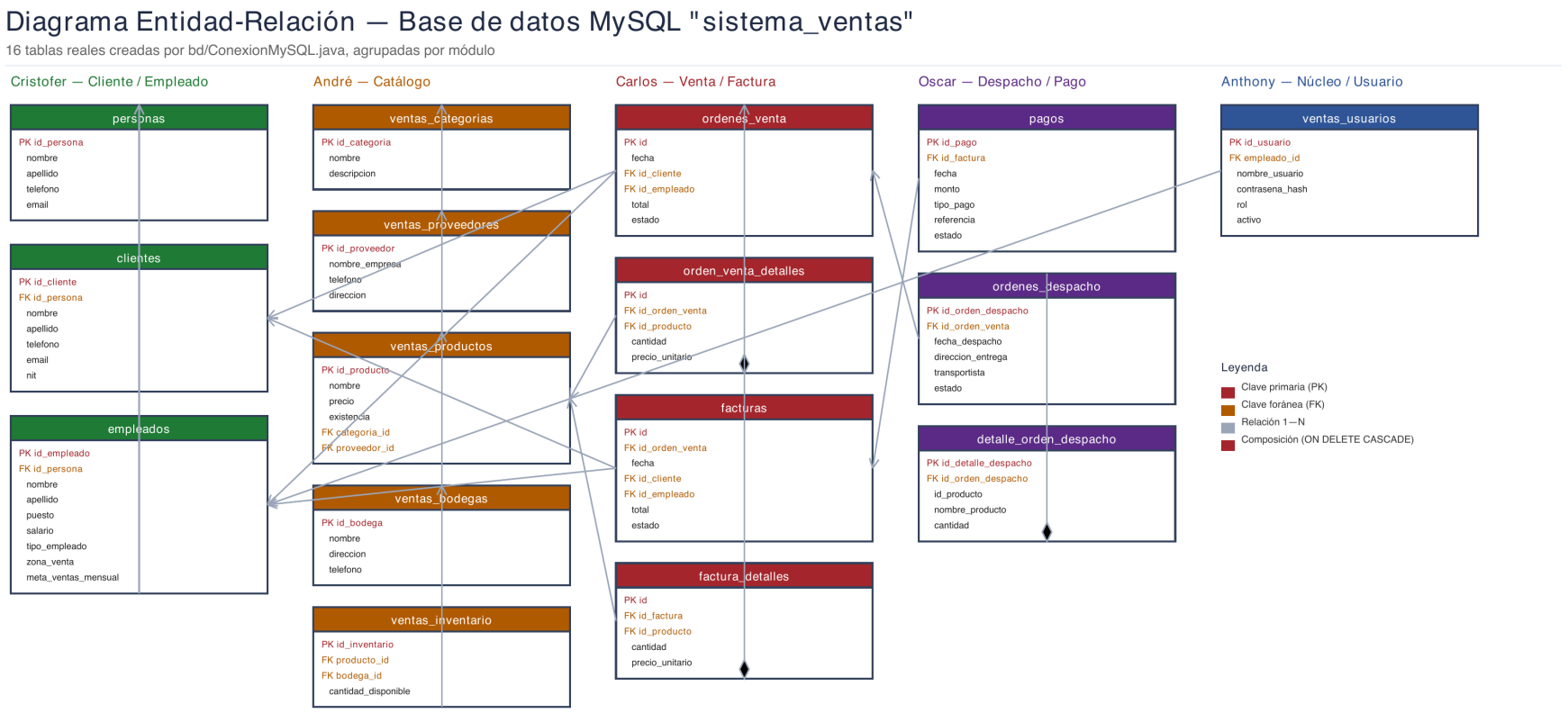
3. Organización del código (paquetes Java)

Todos deben usar exactamente esta misma estructura de carpetas desde el primer día para evitar conflictos al integrar el trabajo de los 5 integrantes:

```
src/main/java/gt/edu/umg/sistema/ventas/
 ├── bd/                    → Anthony (conexión PostgreSQL)
 │    ├── ConexionPostgreSQL.java
 │    └── ConexionException.java
 │
 ├── modelo/                → Entidades (repartidas entre todos, ver tabla)
 │    ├── Persona.java              (abstracta)        → Anthony
 │    ├── Usuario.java                                  → Anthony
 │    ├── Cliente.java                                  → Cristofer
```

8. Diagrama Entidad-Relación (ER)

Corresponde a la base de datos MySQL "sistema_ventas" ya creada por bd/ConexionMySQL.java: 16 tablas, agrupadas por módulo.



Nota:
los nombres reales de algunas tablas difieren de los propuestos en las secciones 6 y 7 de este documento (ver bd/ConexionMySQL.java).

```
│   ├── Empleado.java                                  → Cristofer
│   ├── Vendedor.java            (extends Empleado)    → Cristofer
│   ├── Proveedor.java                                 → André
│   ├── Categoria.java                                 → André
│   ├── Producto.java                                  → André
│   ├── Inventario.java                                → André
│   ├── Bodega.java                                     → André
│   ├── Factura.java                                    → Carlos
│   ├── DetalleFactura.java                             → Carlos
│   ├── OrdenVenta.java                                 → Carlos
│   ├── DetalleOrdenVenta.java                          → Carlos
│   ├── OrdenDespacho.java                              → Oscar
│   ├── DetalleOrdenDespacho.java                       → Oscar
│   ├── Pago.java                                       → Oscar
│   └── pago/
│       ├── IMetodoPago.java                           → Oscar
│       ├── PagoEfectivo.java                          → Oscar
│       ├── PagoTarjeta.java                            → Oscar
│       └── PagoTransferencia.java                     → Oscar
│
├── dao/                  → Interfaces + implementación (una pareja por entidad)
│   ├── UsuarioDAO.java / UsuarioDAOImpl.java             → Anthony
│   ├── ClienteDAO.java / ClienteDAOImpl.java             → Cristofer
│   ├── EmpleadoDAO.java / EmpleadoDAOImpl.java           → Cristofer
│   ├── ProductoDAO.java / ProductoDAOImpl.java           → André
│   ├── BodegaDAO.java / BodegaDAOImpl.java               → André
│   ├── FacturaDAO.java / FacturaDAOImpl.java             → Carlos
│   ├── OrdenVentaDAO.java / OrdenVentaDAOImpl.java       → Carlos
│   ├── OrdenDespachoDAO.java / OrdenDespachoDAOImpl.java → Oscar
│   └── PagoDAO.java / PagoDAOImpl.java                   → Oscar
│
├── controlador/          → Un controlador por módulo (conecta Vista ↔ DAO)
│   ├── UsuarioController.java                            → Anthony
│   ├── ClienteController.java                             → Cristofer
│   ├── EmpleadoController.java                            → Cristofer
│   ├── ProductoController.java                            → André
│   ├── BodegaController.java                              → André
│   ├── FacturaController.java                              → Carlos
│   ├── OrdenVentaController.java                           → Carlos
│   ├── OrdenDespachoController.java                       → Oscar
│   └── PagoController.java                                 → Oscar
│
├── vista/                → Pantallas Swing (JFrame)
│   ├── FrmLogin.java             → Anthony  (pantalla inicial, valida Usuario)
│   ├── FrmMenuPrincipal.java     → Anthony  (muestra opciones segun el rol)
│   ├── FrmClientes.java          → Cristofer
│   ├── FrmEmpleados.java         → Cristofer (incluye alta de Vendedor)
│   ├── FrmProductos.java         → André
│   ├── FrmBodegas.java           → André
│   ├── FrmOrdenVenta.java        → Carlos   (prototipo de "hacer pedido", igual a lo que sera la app movil)
│   ├── FrmFacturacion.java       → Carlos
│   ├── FrmOrdenDespacho.java     → Oscar
│   └── FrmPago.java              → Oscar
│
└── SistemaVentas.java    → Anthony (clase main, abre FrmLogin)
```

4. Asignación detallada por integrante

Integrante	Clases del modelo	Archivos a crear	Depende de
Anthony	Persona (abstracta), Usuario	bd/ConexionPostgreSQL, bd/ConexionException, modelo/Persona, modelo/Usuario, dao/UsuarioDAO(+Impl), controlador/UsuarioController, vista/FrmLogin, vista/FrmMenuPrincipal, SistemaVentas.java	— (es la base, debe entregarse primero)
Cristofer	Cliente, Empleado, Vendedor	modelo/Cliente, modelo/Empleado, modelo/Vendedor, dao/ClienteDAO(+Impl), dao/EmpleadoDAO(+Impl), controlador/ClienteController, controlador/EmpleadoController, vista/FrmClientes, vista/FrmEmpleados	Persona y Usuario (Anthony)
André	Producto, Categoria, Inventario, Proveedor, Bodega	modelo/Producto, modelo/Categoria, modelo/Inventario, modelo/Proveedor, modelo/Bodega, dao/ProductoDAO(+Impl),	— (independiente)

		dao/BodegaDAO(+Impl), controlador/ProductoController, controlador/BodegaController, vista/FrmProductos, vista/FrmBodegas	
Carlos	Factura, DetalleFactura, OrdenVenta, DetalleOrdenVenta	modelo/Factura, modelo/DetalleFactura, modelo/OrdenVenta, modelo/DetalleOrdenVenta, dao/FacturaDAO(+Impl), dao/OrdenVentaDAO(+Impl), controlador/FacturaController, controlador/OrdenVentaController, vista/FrmOrdenVenta, vista/FrmFacturacion	Cliente y Vendedor (Cristofer), Producto (André)
Oscar	Pago, IMetodoPago (+3 clases), OrdenDespacho, DetalleOrdenDespacho	modelo/Pago, modelo/pago/IMetodoPago(+3 clases), modelo/OrdenDespacho, modelo/DetalleOrdenDespacho, dao/PagoDAO(+Impl), dao/OrdenDespachoDAO(+Impl), controlador/PagoController, controlador/OrdenDespachoController, vista/FrmOrdenDespacho, vista/FrmPago	OrdenVenta (Carlos), Bodega (André)

5. Orden de trabajo sugerido

Paso 1 — Acuerdo inicial (todos juntos, primer día): definan en una sola sesión los atributos exactos de `Persona`, `Cliente`, `Empleado` y `Producto` (nombre y tipo de cada campo). Esto evita que Carlos y Oscar tengan que reescribir código después.

Paso 2 — Base del proyecto: Anthony sube `Persona`, `Usuario`, la conexión a PostgreSQL y el proyecto Maven vacío al repositorio compartido. Sin esto nadie más puede compilar.

Paso 3 — Desarrollo en paralelo: Cristofer y André empiezan primero porque nadie depende de ellos (Cliente/Empleado/Vendedor y Producto/Bodega son independientes). Carlos necesita que Cristofer y André avancen antes de programar `OrdenVenta`; mientras tanto puede ir haciendo la pantalla y dejar clases *stub* de prueba. Oscar necesita que Carlos tenga lista `OrdenVenta` antes de programar `OrdenDespacho`, pero puede adelantar `Pago` e `IMetodoPago` sin depender de nadie.

Paso 4 — Integración: Anthony une todas las ramas en `FrmMenuPrincipal` (mostrando las opciones según el `rol` del `Usuario` que inició sesión) y prueba el flujo completo de principio a fin: `Cliente + Vendedor → OrdenVenta → OrdenDespacho → Factura → Pago`.

Paso 5 — Prueba general del equipo: todos prueban el programa juntos, revisan que los datos se guarden correctamente en PostgreSQL (incluyendo que el stock se descuente solo una vez, no dos) y corrigen errores de integración antes de la entrega.

> **Pensando en la futura app móvil:** la pantalla `FrmOrdenVenta` que hace Carlos debe programarse como si ya fuera una API — es decir, el `OrdenVentaController.crearPedido(...)` no debe depender de Swing para nada de su lógica. Así, cuando más adelante se construya la app móvil, esa misma lógica de negocio se puede reutilizar detrás de un servicio REST sin tener que reescribirla.

6. Especificación técnica detallada por integrante

Esta sección indica, clase por clase, exactamente qué atributos, métodos, tabla de base de datos y componentes de pantalla debe crear cada integrante. Todos los atributos son `private` con sus `get/set` correspondientes (no se listan por espacio).

Anthony — Núcleo, base de datos, Usuarios e integración

Clase	Atributos propios	Métodos propios
`modelo/Persona` abstracta	`id:int, nombre:String, apellido:String, telefono:String, direccion:String`	`mostrarInformacion(): String`
`modelo/Usuario`	`idUsuario:int, nombreUsuario:String, contrasenaHash:String, rol:String, activo:boolean, empleado:Empleado`	`autenticar(pwd:String): boolean`

```
-- Tabla Usuario (login del sistema; el vendedor tambien inicia sesion aqui desde su celular)
CREATE TABLE IF NOT EXISTS ventas_usuarios (
  id_usuario INT NOT NULL PRIMARY KEY,
  empleado_id INT NOT NULL UNIQUE,
  nombre_usuario VARCHAR(50) NOT NULL UNIQUE,
  contrasena_hash VARCHAR(255) NOT NULL,
  rol VARCHAR(30) NOT NULL,          -- 'ADMIN' | 'VENDEDOR' | 'BODEGA' | 'CAJA'
  activo BOOLEAN NOT NULL DEFAULT TRUE,
  CONSTRAINT fk_usuario_empleado FOREIGN KEY (empleado_id) REFERENCES ventas_empleados(id)
);
```

Archivo	Qué debe contener
`bd/ConexionPostgreSQL.java`	`getConnection(): Connection` y `crearTablas()` con secciones comentadas para que cada compañero pegue su SQL (ver más abajo).
`bd/ConexionException.java`	Excepción simple que envuelve `SQLException`.
`dao/UsuarioDAO.java` (interfaz)	`guardar(u: Usuario)`, `buscarPorUsuario(nombre: String): Usuario`
`dao/UsuarioDAOImpl.java`	Guarda la contraseña **siempre hasheada** (usar `BCrypt` o al menos `SHA-256`, nunca texto plano)
`controlador/UsuarioController.java`	`login(nombreUsuario:String, pwd:String): Usuario` (retorna `null` si falla)
`vista/FrmLogin.java`	Campos usuario/contraseña · botón "Iniciar sesión" · si es correcto, abre `FrmMenuPrincipal` pasándole el `Usuario` autenticado
`vista/FrmMenuPrincipal.java`	Muestra solo las opciones que le corresponden al `rol` del `Usuario` (ej. un Vendedor solo ve "Hacer Pedido", no ve Facturación ni Bodega)
`SistemaVentas.java`	Clase `main` que ahora abre `FrmLogin` (ya no `FrmMenuPrincipal` directamente)
`pom.xml`	Dependencia `org.postgresql:postgresql` (y opcionalmente `org.mindrot:jbcrypt` para hashear contraseñas)

Tareas de coordinación: crear el repositorio Git, invitar a los 4 compañeros, revisar cada Pull Request, y al final correr el flujo completo: login → pedido → despacho → factura → pago.

Cristofer Módulo Cliente, Empleado y Vendedor

Clase	Atributos propios	Métodos propios
`modelo/Cliente` extends Persona	`nit:String, correo:String, fechaRegistro:LocalDate`	`toString()`
`modelo/Empleado` extends Persona	`codigoEmpleado:String, puesto:String, salario:double`	`toString()`
`modelo/Vendedor` extends Empleado	`zonaVenta:String, metaVentasMensual:double`	`toString()`

> **Cómo se guarda Vendedor:** en vez de crear una tabla nueva, usen **una sola tabla** para todo el personal (patrón "tabla por jerarquía"), con una columna `tipo_empleado` que diga si es `'VENDEDOR'` o `'GENERAL'`. Es más simple de programar en JDBC que separar en dos tablas con join.

```
-- Tabla Cliente
CREATE TABLE IF NOT EXISTS ventas_clientes (
  id INT NOT NULL PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL,
  apellido VARCHAR(100) NOT NULL,
  nit VARCHAR(50) NOT NULL,
  correo VARCHAR(120),
  telefono VARCHAR(50),
  direccion VARCHAR(200),
  fecha_registro DATE
);

-- Tabla Empleado (incluye a los Vendedores en las mismas filas)
CREATE TABLE IF NOT EXISTS ventas_empleados (
  id INT NOT NULL PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL,
  apellido VARCHAR(100) NOT NULL,
  codigo_empleado VARCHAR(50) NOT NULL UNIQUE,
  puesto VARCHAR(100),
  salario DECIMAL(10,2),
  telefono VARCHAR(50),
  direccion VARCHAR(200),
  tipo_empleado VARCHAR(20) NOT NULL DEFAULT 'GENERAL',  -- 'GENERAL' | 'VENDEDOR'
  zona_venta VARCHAR(100),            -- solo si tipo_empleado = 'VENDEDOR'
  meta_ventas_mensual DECIMAL(12,2)   -- solo si tipo_empleado = 'VENDEDOR'
);
```

Archivo	Métodos a implementar
`dao/ClienteDAO.java` (interfaz)	`guardar(Cliente)`, `listar(): List<Cliente>`, `buscarPorId(int): Cliente`
`dao/ClienteDAOImpl.java`	Implementa con `PreparedStatement` (usar `INSERT ... ON CONFLICT (id) DO UPDATE SET ...` para que "Guardar" también sirva de "Editar")
`dao/EmpleadoDAO.java` / `EmpleadoDAOImpl.java`	`guardar`, `listar`, y `listarVendedores(): List<Vendedor>` (filtra por `tipo_empleado='VENDEDOR'`) — **Carlos necesita este último método**
`controlador/ClienteController.java`	Delega al DAO: `guardar`, `listar`
`controlador/EmpleadoController.java`	Delega a `EmpleadoDAO`; expone `listarVendedores()`
`vista/FrmClientes.java`	Campos: nombre, apellido, nit, correo, teléfono, dirección · botones Guardar/Limpiar · JTable con la lista
`vista/FrmEmpleados.java`	Igual que FrmClientes, más un JCheckBox "Es vendedor" que al marcarse muestra los campos zona de venta y meta mensual

André Módulo Producto, Categoría, Inventario, Proveedor y Bodega

Clase	Atributos propios	Métodos propios
`modelo/Categoria`	`idCategoria:int, nombre:String, descripcion:String`	`toString()`
`modelo/Proveedor`	`idProveedor:int, nombreEmpresa:String, telefono:String, direccion:String`	`toString()`
`modelo/Producto`	`idProducto:int, nombre:String, descripcion:String, precio:double, existencia:int, categoria:Categoria, proveedor:Proveedor`	`hayExistencia(cant:int): boolean`
`modelo/Bodega` (nueva)	`idBodega:int, nombre:String, direccion:String, telefono:String`	`toString()`
`modelo/Inventario`	`idInventario:int, producto:Producto, bodega:Bodega, cantidadDisponible:int`	`actualizarStock(cant:int)`

```
CREATE TABLE IF NOT EXISTS ventas_categorias (
  id_categoria INT NOT NULL PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL,
  descripcion VARCHAR(200)
);

CREATE TABLE IF NOT EXISTS ventas_proveedores (
  id_proveedor INT NOT NULL PRIMARY KEY,
  nombre_empresa VARCHAR(150) NOT NULL,
  telefono VARCHAR(50),
  direccion VARCHAR(200)
);

CREATE TABLE IF NOT EXISTS ventas_productos (
  id_producto INT NOT NULL PRIMARY KEY,
  nombre VARCHAR(120) NOT NULL,
  descripcion VARCHAR(200),
  precio DECIMAL(12,2) NOT NULL,
  existencia INT NOT NULL,
  categoria_id INT,
  proveedor_id INT,
  CONSTRAINT fk_producto_categoria FOREIGN KEY (categoria_id) REFERENCES ventas_categorias(id_categoria),
  CONSTRAINT fk_producto_proveedor FOREIGN KEY (proveedor_id) REFERENCES ventas_proveedores(id_proveedor)
);

-- Bodega: nueva tabla (antes el inventario solo tenia un campo de texto)
CREATE TABLE IF NOT EXISTS ventas_bodegas (
  id_bodega INT NOT NULL PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL,
  direccion VARCHAR(200),
  telefono VARCHAR(50)
);

CREATE TABLE IF NOT EXISTS ventas_inventario (
  id_inventario INT NOT NULL PRIMARY KEY,
  producto_id INT NOT NULL,
  bodega_id INT NOT NULL,
  cantidad_disponible INT NOT NULL,
  CONSTRAINT fk_inventario_producto FOREIGN KEY (producto_id) REFERENCES ventas_productos(id_producto),
  CONSTRAINT fk_inventario_bodega FOREIGN KEY (bodega_id) REFERENCES ventas_bodegas(id_bodega)
);
```

Archivo	Métodos a implementar
`dao/ProductoDAO.java` (interfaz)	`guardar`, `listar`, `buscarPorId`, `actualizarExistencia(id:int, cant:int)` — **lo usarán Carlos y Oscar**, avísales cuando esté listo
`dao/ProductoDAOImpl.java`	Incluir `listarCategorias()` y `listarProveedores()` auxiliares para llenar combos
`dao/BodegaDAO.java` / `BodegaDAOImpl.java` (nuevo)	`guardar(Bodega)`, `listar(): List<Bodega>`
`controlador/ProductoController.java`	Delega a ProductoDAO; expone `listarCategorias()`, `listarProveedores()`
`controlador/BodegaController.java` (nuevo)	Delega a BodegaDAO
`vista/FrmProductos.java`	Campos producto · combos Categoría/Proveedor · JTable con existencia (resaltar en rojo si < 5)
`vista/FrmBodegas.java` (nueva)	CRUD simple: nombre, dirección, teléfono · JTable de bodegas

Carlos Módulo Facturación y Orden de Venta (el "pedido")

> **Este es el módulo que a futuro se vuelve la app móvil.** El `Vendedor` hace el pedido (`OrdenVenta`) desde el celular; ahora lo simulan con una pantalla Swing, pero la lógica del controlador debe quedar separada de Swing para poder reutilizarla después.

Clase	Atributos propios	Métodos propios
`modelo/OrdenVenta` (nueva)	`idOrdenVenta:int, fecha:LocalDate, cliente:Cliente, vendedor:Vendedor, estado:String, detalles:List<DetalleOrdenVenta>`	`calcularTotal(): double, agregarDetalle(...)`
`modelo/DetalleOrdenVenta` (nueva)	`producto:Producto, cantidadSolicitada:int, precioUnitario:double`	`calcularSubtotal(): double`
`modelo/Factura`	`idFactura:int, fecha:LocalDate, cliente:Cliente, empleado:Empleado, ordenVenta:OrdenVenta, subtotal:double, impuesto:double, total:double, estado:String, detalles:List<DetalleFactura>`	`calcularTotal(): double`
`modelo/DetalleFactura`	`producto:Producto, cantidad:int, precioUnitario:double`	`calcularSubtotal(): double`

```
-- Orden de Venta = el "pedido" (estado: Pendiente / Despachada / Facturada / Cancelada)
CREATE TABLE IF NOT EXISTS ventas_ordenes_venta (
  id_orden_venta INT NOT NULL PRIMARY KEY,
  fecha DATE NOT NULL,
  cliente_id INT NOT NULL,
  vendedor_id INT NOT NULL,
  estado VARCHAR(30) NOT NULL DEFAULT 'Pendiente',
  CONSTRAINT fk_ov_cliente FOREIGN KEY (cliente_id) REFERENCES ventas_clientes(id),
  CONSTRAINT fk_ov_vendedor FOREIGN KEY (vendedor_id) REFERENCES ventas_empleados(id)
);

CREATE TABLE IF NOT EXISTS ventas_orden_venta_detalles (
  id_detalle BIGSERIAL PRIMARY KEY,
  orden_venta_id INT NOT NULL,
  producto_id INT NOT NULL,
  cantidad_solicitada INT NOT NULL,
  precio_unitario DECIMAL(12,2) NOT NULL,
  CONSTRAINT fk_ovd_orden FOREIGN KEY (orden_venta_id) REFERENCES ventas_ordenes_venta(id_orden_venta) ON DELETE CASCADE,
  CONSTRAINT fk_ovd_producto FOREIGN KEY (producto_id) REFERENCES ventas_productos(id_producto)
);

-- Facturas: se agrega la columna que la conecta con la orden de venta que la origino
CREATE TABLE IF NOT EXISTS ventas_facturas (
  id_factura INT NOT NULL PRIMARY KEY,
  orden_venta_id INT,                -- puede ser NULL si se factura directo, sin pedido previo
  fecha DATE NOT NULL,
  cliente_id INT NOT NULL,
  empleado_id INT NOT NULL,
  subtotal DECIMAL(12,2) NOT NULL,
  impuesto DECIMAL(12,2) NOT NULL,
  total DECIMAL(12,2) NOT NULL,
  estado VARCHAR(30) NOT NULL DEFAULT 'Pendiente',
  CONSTRAINT fk_factura_orden FOREIGN KEY (orden_venta_id) REFERENCES ventas_ordenes_venta(id_orden_venta),
  CONSTRAINT fk_factura_cliente FOREIGN KEY (cliente_id) REFERENCES ventas_clientes(id),
  CONSTRAINT fk_factura_empleado FOREIGN KEY (empleado_id) REFERENCES ventas_empleados(id)
);

CREATE TABLE IF NOT EXISTS ventas_factura_detalles (
  id_detalle BIGSERIAL PRIMARY KEY,
  factura_id INT NOT NULL,
  producto_id INT NOT NULL,
  cantidad INT NOT NULL,
  precio_unitario DECIMAL(12,2) NOT NULL,
  CONSTRAINT fk_detalle_factura FOREIGN KEY (factura_id) REFERENCES ventas_facturas(id_factura) ON DELETE CASCADE,
  CONSTRAINT fk_detalle_producto FOREIGN KEY (producto_id) REFERENCES ventas_productos(id_producto)
);
```

Archivo	Métodos a implementar
`dao/OrdenVentaDAO.java` (interfaz) (nuevo)	`guardar(OrdenVenta)`, `listarPendientes(): List<OrdenVenta>`, `actualizarEstado(id:int, estado:String)`
`dao/OrdenVentaDAOImpl.java`	`listarPendientes()` es la que usará Oscar para saber qué despachar
`dao/FacturaDAO.java` / `FacturaDAOImpl.java`	**Transaccional**: inserta factura + detalles, descuenta existencia vía `ProductoDAO.actualizarExistencia(...)`, y marca

	`ventas_ordenes_venta.estado='Facturada'` si venía de un pedido
`controlador/OrdenVentaController.java` (nuevo)	`crearPedido(Cliente, Vendedor, List<DetalleOrdenVenta>): OrdenVenta` — **sin código de Swing adentro**, para reutilizarla en la futura app móvil
`controlador/FacturaController.java`	`generarFacturaDesdeOrden(OrdenVenta): Factura`, más `calcularTotales(...)`
`vista/FrmOrdenVenta.java` (nueva)	Combo Cliente, combo Vendedor, combo Producto + cantidad → tabla de detalle → botón "Enviar Pedido" (queda en estado Pendiente)
`vista/FrmFacturacion.java`	Lista de `OrdenVenta` ya despachadas, listas para facturar · botón "Generar Factura" · tabla de historial de facturas

> Antes de empezar, Carlos necesita que Cristofer tenga `Cliente` y `Vendedor`, y que André tenga `Producto`.

Oscar Módulo Pagos y Orden de Despacho

Clase	Atributos propios	Métodos propios
modelo/OrdenDespacho (nueva)	idOrdenDespacho:int, ordenVenta:OrdenVenta, bodega:Bodega, responsable:Empleado, fechaDespacho:LocalDate, estado:String, detalles:List<DetalleOrdenDespacho>	confirmarDespacho(): boolean
modelo/DetalleOrdenDespacho (nueva)	producto:Producto, cantidadDespachada:int	—
modelo/Pago	idPago:int, factura:Factura, monto:double, fechaPago:LocalDate, metodoPago:IMetodoPago	procesarPago(): boolean
modelo/pago/IMetodoPago (interfaz)	—	procesarPago(monto:double): boolean
modelo/pago/PagoEfectivo	montoRecibido:double	calcularCambio(): double
modelo/pago/PagoTarjeta	numeroTarjeta:String, tipoTarjeta:String	validarTarjeta(): boolean
modelo/pago/PagoTransferencia	numeroCuenta:String, banco:String	validarTransferencia(): boolean

```
-- Orden de Despacho: lo que la Bodega prepara a partir del pedido
CREATE TABLE IF NOT EXISTS ventas_ordenes_despacho (
  id_orden_despacho INT NOT NULL PRIMARY KEY,
  orden_venta_id INT NOT NULL,
  bodega_id INT NOT NULL,
  empleado_id INT NOT NULL,          -- bodeguero responsable
  fecha_despacho DATE,
  estado VARCHAR(30) NOT NULL DEFAULT 'Pendiente',   -- Pendiente / Preparado / Entregado
  CONSTRAINT fk_od_orden FOREIGN KEY (orden_venta_id) REFERENCES ventas_ordenes_venta(id_orden_venta),
  CONSTRAINT fk_od_bodega FOREIGN KEY (bodega_id) REFERENCES ventas_bodegas(id_bodega),
  CONSTRAINT fk_od_empleado FOREIGN KEY (empleado_id) REFERENCES ventas_empleados(id)
);

CREATE TABLE IF NOT EXISTS ventas_orden_despacho_detalles (
  id_detalle BIGSERIAL PRIMARY KEY,
  orden_despacho_id INT NOT NULL,
  producto_id INT NOT NULL,
  cantidad_despachada INT NOT NULL,
  CONSTRAINT fk_odd_orden FOREIGN KEY (orden_despacho_id) REFERENCES ventas_ordenes_despacho(id_orden_despacho) ON DELETE CASCADE,
  CONSTRAINT fk_odd_producto FOREIGN KEY (producto_id) REFERENCES ventas_productos(id_producto)
);

-- Pago (igual que antes)
CREATE TABLE IF NOT EXISTS ventas_pagos (
  id_pago INT NOT NULL PRIMARY KEY,
  factura_id INT NOT NULL,
  monto DECIMAL(12,2) NOT NULL,
  fecha_pago DATE NOT NULL,
  tipo_metodo VARCHAR(20) NOT NULL,
  detalle_metodo VARCHAR(150),
  CONSTRAINT fk_pago_factura FOREIGN KEY (factura_id) REFERENCES ventas_facturas(id_factura)
);
```

Archivo	Métodos a implementar
dao/OrdenDespachoDAO.java (interfaz) (nuevo)	guardar(OrdenDespacho), listar(): List<OrdenDespacho>
dao/OrdenDespachoDAOImpl.java	Al confirmar el despacho: descuenta existencia vía ProductoDAO.actualizarExistencia(...) y llama a OrdenVentaDAO.actualizarEstado(id, "Despachada")
dao/PagoDAO.java / PagoDAOImpl.java	Al guardar un pago, actualiza ventas_facturas.estado = 'Pagada'
controlador/OrdenDespachoController.java (nuevo)	generarDespacho(OrdenVenta, Bodega, Empleado): OrdenDespacho
controlador/PagoController.java	procesarPago(Pago): boolean
vista/FrmOrdenDespacho.java (nueva)	Lista de OrdenVenta pendientes (llama a OrdenVentaDAO.listarPendientes() de Carlos) · combo Bodega · tabla para confirmar cantidades reales · botón "Confirmar Despacho"
vista/FrmPago.java	Combo de facturas pendientes · radio buttons Efectivo/Tarjeta/Transferencia · botón "Procesar Pago"

Oscar depende de que Carlos tenga lista `OrdenVenta` y de que André tenga lista `Bodega`. Mientras tanto, puede adelantar `IMetodoPago` y sus 3 estrategias, que no dependen de nadie.

7. Control de versiones: nombre del commit

El **primer commit** lo sube Anthony, con la estructura base (paquetes vacíos, `pom.xml`, `Persona`, `Usuario`, `ConexionPostgreSQL`). Debe llamarse:

```
git init
git add .
git commit -m "feat: estructura base del proyecto (MVC + DAO), login y conexion a PostgreSQL"
git branch -M main
git remote add origin https://github.com/usuario/sistema-ventas.git
git push -u origin main
```

Para los siguientes commits (de cada integrante), usen esta convención — así se entiende de inmediato quién tocó qué sin abrir el código:

```
tipo(modulo): descripcion corta en minuscula, sin punto final

Tipos:  feat     → funcionalidad nueva
        fix      → corrección de un error
        refactor → mejora de código sin cambiar comportamiento
        docs     → documentación (este mismo PDF, comentarios, README)
```

Integrante	Rama sugerida	Ejemplo de commit
Cristofer	`feature/cliente-empleado-vendedor`	`feat(vendedor): agrega Vendedor como especializacion de Empleado`
André	`feature/producto-bodega`	`feat(bodega): modelo Bodega y relacion con Inventario`
Carlos	`feature/orden-venta-facturacion`	`feat(orden-venta): registro de pedidos desde el vendedor`
Oscar	`feature/pagos-despacho`	`feat(orden-despacho): confirmacion de despacho desde bodega`
Anthony	`main` (recibe los merges)	`feat(usuario): login con roles y control de acceso al menu`

Cada quien hace *commit* en su propia rama y abre un *Pull Request* hacia `main` cuando su módulo esté listo; Anthony revisa y aprueba antes de fusionar.

Documento generado como base de trabajo. El diagrama de clases y la organización aquí definidos servirán de referencia durante toda la implementación del proyecto.